LAPORAN TUGAS KELOMPOK
IMPLEMENTASI SISTEM CLIENT–SERVER
S1 ELEKTRONIKA DAN INSTRUMENTASI

MATA KULIAH

KAPITA SELEKTA SISTEM KOMPUTER JARINGAN

KELAS IEEL

Oleh:

Vytis Rabbani Rex (23/511414/PA/21789)

Aufa Akmal Bunaya (23/515767/PA/22027)

Ahmad Firdaus Zen Omar Idrus (23/521171/PA/22406)

Nawal Arifah Herman (23/523349/PA/22520)

Ihsan Hammam (24/532900/PA/22551)

DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

1. Penjelasan Arsitektur Sistem

Sistem yang dikembangkan dalam tugas ini mengadopsi arsitektur *client–server* berbasis protokol TCP (*Transmission Control Protocol*). Arsitektur ini merupakan model komunikasi terdistribusi di mana server berperan sebagai penyedia layanan (*service provider*), sedangkan client berperan sebagai peminta layanan (*service requester*).

1.1. Arsitektur Server

Server dirancang untuk:

1. Membuat *socket* menggunakan protokol TCP.
2. Melakukan binding terhadap alamat IP dan nomor port tertentu.
3. Masuk ke mode *listening* untuk menunggu koneksi masuk.
4. Menerima koneksi dari client.
5. Menghasilkan soal aritmatika secara acak.
6. Menerima dan memvalidasi jawaban client.
7. Mengirimkan respons hasil evaluasi.
8. Menutup sesi komunikasi dan kembali ke mode *listening*.

Server menggunakan alamat 0.0.0.0, yang berarti menerima koneksi dari seluruh antarmuka jaringan yang tersedia pada sistem operasi. Nomor port yang digunakan adalah 5012, sesuai ketentuan penyesuaian kelompok. Struktur server dirancang menggunakan perulangan tak hingga (*while True*) agar layanan tetap tersedia setelah satu sesi komunikasi selesai.

1.2. Arsitektur Client

Client bertugas untuk:

1. Membuat socket TCP.
2. Menghubungkan diri ke alamat IP dan port server.
3. Menerima pesan sambutan dan soal.
4. Mengirim jawaban ke server.
5. Menerima hasil evaluasi.
6. Menutup koneksi setelah proses selesai.

Client hanya aktif selama sesi komunikasi berlangsung dan tidak mempertahankan koneksi setelah program dihentikan.

1.3. Karakteristik Komunikasi

Komunikasi yang digunakan memiliki karakteristik:

- *Connection-oriented*, karena TCP mengharuskan proses *handshake* sebelum pertukaran data.
- *Reliable*, karena TCP menjamin data sampai secara utuh dan berurutan.
- Menggunakan format data berbasis string yang dikodekan dalam UTF-8.

Model komunikasi ini mencerminkan implementasi dasar layanan jaringan terstruktur pada tingkat aplikasi.

2. Flow Komunikasi Data

Alur komunikasi antara client dan server berlangsung sebagai berikut:

1. Server dijalankan dan melakukan proses ‘bind()’ terhadap port yang ditentukan.
2. Server memasuki mode ‘listen()’ dan menunggu koneksi.
3. Client dijalankan dan melakukan ‘connect()’ ke alamat server.
4. Server menerima koneksi menggunakan ‘accept()’.
5. Server mengirimkan pesan sambutan kepada client.
6. Server menghasilkan dua angka acak dan membentuk soal aritmatika.
7. Server mengirim soal ke *client*.
8. *Client* menerima soal dan menampilkan kepada pengguna.
9. Pengguna memasukkan jawaban melalui terminal.
10. Client mengirim jawaban ke server.
11. Server menerima jawaban dan melakukan validasi.
12. Server mengirim respons berupa:
    - "BENAR"
    - "SALAH. Jawaban benar: ..."
    - "Input tidak valid"
13. Jika *client* mengirimkan perintah "exit", server mengakhiri sesi.
14. *Client* menutup koneksi.
15. Server kembali ke mode *listening*.

Alur ini menunjukkan adanya siklus komunikasi dua arah (*bidirectional communication*) selama sesi berlangsung.

3. Penjelasan Setiap Fungsi Utama

3.1. Fungsi pada Server

a. ‘socket.socket(AF_INET, SOCK_STREAM)’

Fungsi ini digunakan untuk membuat objek socket berbasis IPv4 (AF_INET) dengan tipe komunikasi TCP (SOCK_STREAM). TCP dipilih karena bersifat connection-oriented dan menjamin keandalan pengiriman data.

Code:

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

b. ‘bind((HOST, PORT))’

Fungsi bind() digunakan untuk mengaitkan socket dengan alamat IP dan nomor port tertentu. Proses ini memungkinkan sistem operasi mengetahui bahwa layanan server berjalan pada port tersebut.

Code:

```
HOST = "0.0.0.0"
PORT = 5012
server_socket.bind((HOST, PORT))
```

c. ‘listen(5)’

Fungsi ini mengaktifkan socket ke mode listening (standby). Parameter 5 menunjukkan jumlah maksimal koneksi yang dapat masuk ke dalam antrean sebelum diproses oleh server.

Code :

```
server_socket.listen(5)
```

d. ‘accept()’

Fungsi accept() digunakan untuk menerima koneksi masuk dari client. Fungsi ini menghasilkan dua nilai:

- Objek socket baru yang khusus digunakan untuk komunikasi dengan client.
- Informasi alamat client (IP dan port)

Code :

```
client_socket, addr = server_socket.accept()
print("Connected by", addr)
```

e. ‘send()’

Fungsi send() digunakan untuk mengirim data ke client. Data yang dikirim harus dalam bentuk byte.

Code:

```
client_socket.send(question.encode("utf-8"))
```

f. ‘recv(1024)’

Fungsi recv() digunakan untuk menerima data dari client dengan ukuran buffer maksimum 1024 byte.

Code:

```
answer = client_socket.recv(1024).decode("utf-8")
```

g. ‘encode("utf-8")’

Metode encode() digunakan untuk mengubah string menjadi representasi byte sebelum dikirim melalui jaringan.

Code:

```
"Selamat datang di Quiz Kalkulator!".encode("utf-8")
```

h. decode("utf-8")

Metode decode() digunakan untuk mengubah data byte yang diterima menjadi string agar dapat diproses lebih lanjut.

Code :

```
answer = client_socket.recv(1024).decode("utf-8")
```

i. ‘random.randint(1, 10)’

Fungsi ini digunakan untuk menghasilkan angka acak dalam rentang tertentu guna membentuk soal aritmatika sederhana.

Code :

```
import random

a = random.randint(1, 10)
b = random.randint(1, 10)
```

```
question = f"Soal: {a} + {b} = ?\n"
```

j. ‘try–except’

Struktur try–except digunakan untuk menangani kemungkinan kesalahan konversi tipe data, khususnya ketika input tidak dapat diubah menjadi bilangan bulat (int).

Code :

```
try:
    if int(answer) == a + b:
        response = "BENAR\n"
    else:
        response = f"SALAH. Jawaban benar: {a+b}\n"
except:
    response = "Input tidak valid\n"
```

3.2. Fungsi pada Client

a. ‘connect((HOST, PORT))’

Fungsi connect() digunakan untuk menginisialisasi koneksi ke server melalui alamat IP dan port yang telah ditentukan.

Code :

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((HOST, PORT))
```

b. ‘recv()’

Fungsi recv() digunakan untuk menerima pesan yang dikirimkan oleh server.

Code :

```
welcome = client_socket.recv(1024).decode("utf-8")
question = client_socket.recv(1024).decode("utf-8")
```

c. ‘input()’

Fungsi input() digunakan untuk menerima jawaban dari pengguna melalui terminal.

Code:

```
answer = input("Jawaban: ")
```

d. ‘send()’

Fungsi send() pada client digunakan untuk mengirim jawaban ke server dalam bentuk byte.

Code:

```
client_socket.send(answer.encode("utf-8"))
```

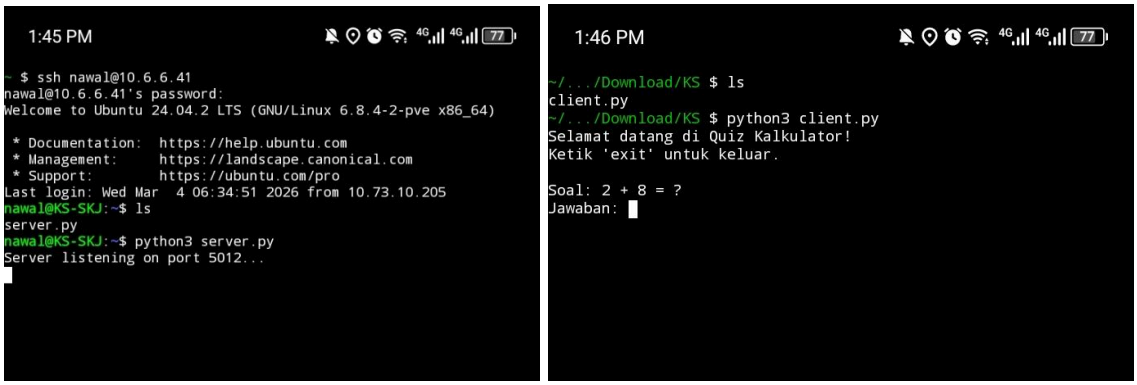
e. ‘close()’

Fungsi close() digunakan untuk menutup koneksi dan membebaskan sumber daya jaringan yang digunakan oleh socket.

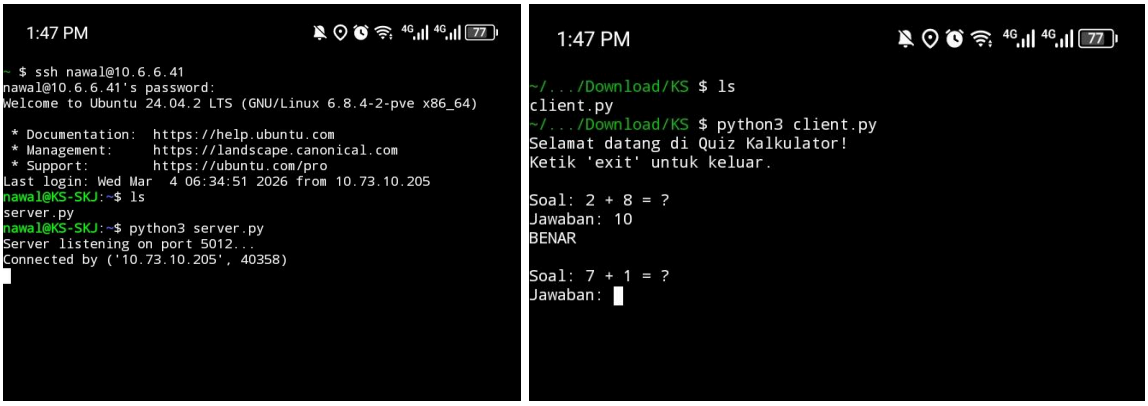
Code:

```
client_socket.close()
```

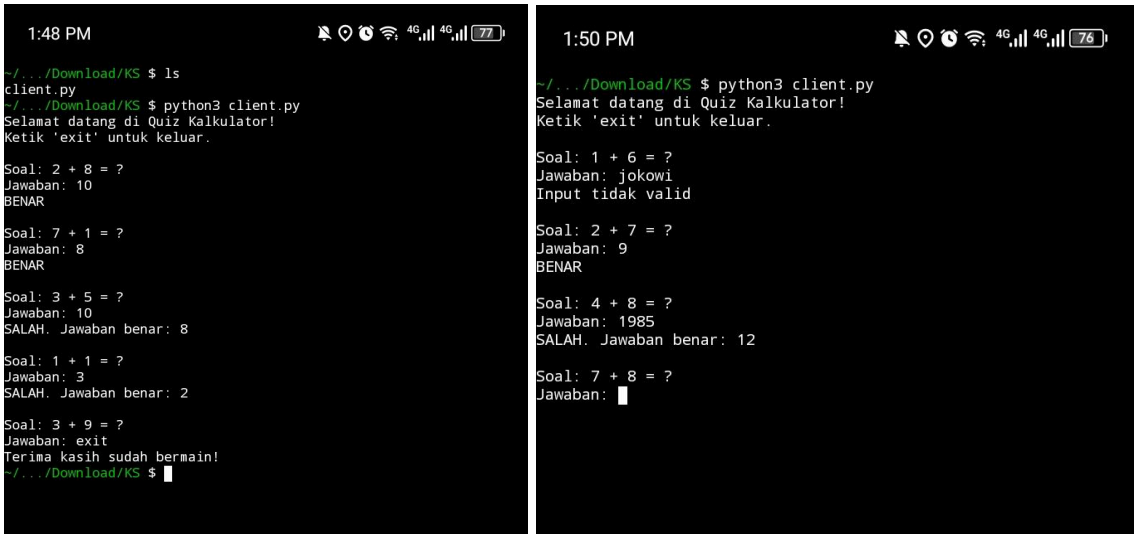
4. Screenshot eksekusi



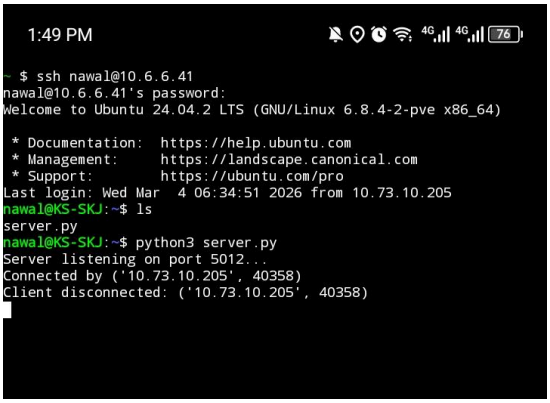
Gambar 1. Login Server dan Menjalankan Script Kode *server.py* dan Client Menjalankan Script Kode *client.py*



Gambar 2. Server Menerima Koneksi *Listen* ke Client dan Client Memainkan Quiz dengan Menjawab Benar



Gambar 3. Client Menjawab Salah dan Mengakhiri Quiz dan Client Menjawab Selain Menggunakan Angka



Gambar 4. Client Memutus Koneksi dan Server Terputus

5. Analisis kesalahan yang mungkin terjadi

Jenis kesalahan	Gejala	Penyebab umum	Mitigasi
IP/Port salah / server tidak aktif (Connection refused/timeout)	Client gagal connect, muncul Connection refused / timed out	Server belum jalan/crash; IP server pada client tidak sesuai; port berbeda; firewall memblokir port	Pastikan server “listening”; samakan IP & port; cek IP aktif; buka port di firewall

Koneksi putus mendadak / jaringan tidak stabil	Server mendeteksi disconnect; recv() kosong; program berhenti tiba-tiba	Client ditutup paksa; jaringan drop; sesi SSH putus	Cek if not data lalu akhiri loop dengan aman; tambah timeout socket; tampilkan pesan “koneksi ditutup”
Server hanya melayani 1 client (blocking)	Client kedua menunggu lama/tidak dilayani	Server menangani koneksi secara serial; loop kuis memblokir accept()	Gunakan threading/multiprocessing atau select agar multi-client paralel (multi-session)
Server “hang” menunggu input	Server terlihat diam, tidak ada respon	Client tidak mengirim jawaban; deadlock alur pesan; tidak ada timeout	Tambah settimeout(); pastikan urutan send/recv konsisten; pakai protokol newline/length-prefix
Encoding/karakter bermasalah	Teks tampil aneh (mojibake) / error decode	Encoding tidak konsisten (bukan UTF-8)	Konsisten pakai UTF-8 pada encode/decode; gunakan .decode('utf-8', errors='replace') bila perlu

6. **Refleksi pemahaman**

Melalui implementasi sistem client–server sederhana berbasis TCP socket, kami memperoleh pemahaman praktis tentang bagaimana komunikasi jaringan bekerja pada level aplikasi, serta bagaimana sistem operasi berperan dalam manajemen koneksi.

Kami memahami bahwa bind() dan listen() mendaftarkan layanan ke OS pada IP/port tertentu, lalu accept() menghasilkan socket baru khusus untuk komunikasi dengan

satu client. Dari sisi client, connect() memicu proses handshake TCP sebelum pertukaran data dimulai. Proses ini menunjukkan bahwa TCP bersifat connection-oriented dan mendukung reliabilitas pengiriman data.

Server berperan sebagai penyedia layanan yang selalu standby, sedangkan client bersifat on-demand untuk meminta layanan. Implementasi kuis aritmatika membantu kami melihat pola request–response secara nyata, yaitu server mengirim soal, client mengirim jawaban, server mengevaluasi lalu mengirim respons.

Dalam praktik jaringan, koneksi bisa putus, input bisa tidak sesuai format, atau port bisa bentrok. Penanganan error yang spesifik (mis. ValueError), timeout untuk mencegah blocking tak terbatas, dan logging aktivitas server akan membuat sistem lebih mudah diuji dan dianalisis.

Jika sistem dikembangkan lebih lanjut, kami akan:

- Menambahkan logging file berisi waktu, IP/port client, soal, jawaban, dan hasil;
- Menambahkan kemampuan multi-client paralel;
- Merapikan protokol pesan (newline/length-prefix);
- Mempertimbangkan keamanan dasar (validasi input, pembatasan ukuran pesan).

7. Pengembangan yang telah dilakukan

Lampiran file kode hasil pengembangan: server.py client.py

Pengembangan tambahan dilakukan dengan threading pada server agar setiap koneksi client ditangani oleh thread terpisah, sehingga server tetap bisa menerima dan melayani multi-client secara paralel tanpa menunggu sesi kuis client lain selesai. Selain itu ditambahkan logging untuk merekam aktivitas penting (waktu, IP/port client, soal, jawaban, dan hasil BENAR/SALAH) ke file *server.log*, sehingga proses debugging dan evaluasi kinerja server menjadi lebih mudah dan terstruktur. Kami juga menerapkan newline protocol (setiap pesan diakhiri *\n* dan dibaca dengan *readline()*), karena TCP bersifat stream sehingga batas pesan tidak terjaga; pendekatan ini mencegah masalah pesan yang tergabung/terpotong ketika kondisi jaringan tidak ideal.